TP2 — Representación vectorial de texto: embeddings y búsqueda semántica

Procesamiento del Lenguaje Natural — TUIA — FCEIA — UNR

Integrantes: Franco Venesia · Marisa Zappala · Agustín Marquardt · Federico Aguirre

Repositorio: `federicomaguirre/procesamiento-lenguaje-natural` (rama `tp2-embeddings`)

Qué hicimos

- **Corpus:** 120 libros de la categoría *Clásico* de Lectulandia. Los 99 del TP1 más 21 nuevos (`src/ampliar_corpus.py`) para cumplir el mínimo de 100.
- **Texto:** todos los modelos ven el mismo texto (título, autores, serie y sinopsis). No incluimos los géneros: así la proyección coloreada por género usa una etiqueta que ningún modelo vio.
- **Cuatro representaciones:**
  - TF-IDF del TP1 (línea de base)
  - Word2Vec propio (skip-gram, 100D, promedio)
  - SBW-vectors-300-min5 (promedio)
  - SBERT `distiluse-base-multilingual-cased-v1` (512D)
- **Persistencia:** dos familias en Supabase/pgvector, `libros_sbw` (300D) y `libros_sbert` (512D). Índice HNSW con `vector_cosine_ops` (el operador es `<=>`), vectores normalizados y validados (sin NaN, inf ni norma 0), y búsqueda con filtro por género resuelta en SQL.
- **Evaluación:** 12 consultas propias (`queries.json`).
- **Parte avanzada:** búsqueda híbrida (full-text de Postgres + SBERT con *Reciprocal Rank Fusion*).

Modelo	P@1	P@3	P@5	P@10
TF-IDF (TP1)	0,42	0,28	0,27	0,21
Word2Vec propio (promedio)	0,33	0,31	0,27	0,20
SBW (promedio)	0,58	0,39	0,33	0,23
SBERT	**0,58**	**0,47**	**0,38**	**0,26**
Full-text Postgres (`ts_rank`)	0,33	0,36	0,30	0,20
Híbrido RRF (full-text + SBERT)	0,50	0,47	0,35	0,24
Azar: esperado / percentil 95	0,04 / 0,17	0,04 / 0,08	0,04 / 0,08	0,04 / 0,07
Techo (mejor P@k posible)	1,00	0,89	0,73	0,42

1. ¿Qué modelo elegiríamos para producción y por qué?

SBERT (`distiluse-base-multilingual-cased-v1`) sobre pgvector, sin la combinación híbrida.

- **Métrica:** es el mejor en P@3, P@5 y P@10, y contra TF-IDF **no pierde en ninguna de las 12 consultas** (gana 5, empata 7). La híbrida no le agregó nada: P@5 de 0,35 contra 0,38.
- **Costo:** los pesos ocupan ~540 MB. Codifica una consulta en ~7 ms en la CPU de una notebook (medido) y no necesita GPU. Los vectores de 512 dimensiones de 120 libros ocupan menos de 250 KB.
- **Privacidad y dependencia de terceros:** corre en nuestro servidor. Las consultas de los usuarios no salen a una API externa, y no dependemos de precios, cuotas ni de que un proveedor discontinúe un modelo.
- **Reproducibilidad:** los pesos son públicos y fijos. Fijando la versión, el mismo texto da siempre el mismo vector, y los embeddings quedan guardados en disco (`embeddings/`).

Dos condiciones antes de ponerlo en producción:

1. **Truncamiento.** El 91 % de los libros supera el límite de 128 tokens y la mitad del texto del corpus nunca llega al modelo. Habría que partir las sinopsis en fragmentos.
2. **Ventaja no concluyente.** La ventaja sobre TF-IDF no es estadísticamente concluyente con 12 consultas (p = 0,06). La elección se apoya más en que nunca pierde y en lo operativo que en el tamaño de la mejora.

Descartamos los promedios de word vectors: en SBW dos libros al azar tienen similitud media 0,89 ± 0,04, así que el espacio discrimina muy poco (los top-5 quedan separados por centésimas). El Word2Vec propio, con 10,6 mil tokens de entrenamiento, aprendió en qué sinopsis aparece cada palabra ("política" → *guerras, españoles, engarzada*, de una frase repetida en 8 sinopsis de Galdós), no qué significa.

2. ¿Cuánto mejor es que TF-IDF? ¿En qué consultas gana y en cuáles no?

- **Cuánto:** SBERT sube la P@5 de 0,27 a 0,38 (+0,12) y la P@1 de 0,42 a 0,58. Son mejoras grandes en términos relativos, pero con 12 consultas el test de permutación pareado da p = 0,06: no podemos descartar que parte sea ruido. Las dos están muy lejos del piso de azar (0,04, percentil 95 = 0,08) y del techo (0,73).
- **Dónde gana:** donde la consulta y la sinopsis no comparten palabras. Medimos el solapamiento léxico de cada consulta con sus libros relevantes:
  - En las 8 consultas con solapamiento menor al 30 %, TF-IDF casi no acierta (P@5 = 0,08) y SBERT y SBW llegan a 0,20.
  - Ejemplo: en "naves espaciales y planetas lejanos", TF-IDF no encuentra nada y SBW trae *El ciclo de Tschai*.
- **Dónde no gana:** en las 4 consultas con mucho solapamiento, TF-IDF (0,65) queda cerca de SBERT (0,75).
  - En las consultas sobre Galdós (Q04) y Sócrates (Q06) empatan: los nombres propios y las palabras exactas alcanzan.
  - El full-text de Postgres incluso mejora a TF-IDF gracias al *stemming* ("detectives" ↔ "detective").
  - En Q10 (comedias satíricas) fallan todos.

3. ¿Qué mide y qué no mide la métrica?

P@k mide **qué fracción de los primeros k resultados anotamos como relevantes**: es lo que ve un usuario en la primera pantalla.

Lo que no mide:

- **El orden dentro del top-k:** acertar en el puesto 1 o en el 5 vale lo mismo. Para eso habría que usar nDCG o MRR.
- **El recall:** en Q04 hay 13 libros relevantes, y una P@5 perfecta no dice nada de los otros 8.
- **Relevancia parcial:** la relevancia es binaria. *Romeo y Julieta* para "amores contrariados en la alta sociedad inglesa" cuenta como 0.
- **Que las consultas no son comparables:** Q07 y Q12 tienen un solo relevante, así que su P@5 máxima es 0,2. Por eso reportamos el techo.
- **La incertidumbre:** con 12 consultas cada una mueve el promedio ~0,08, y diferencias de 0,05 entre modelos están dentro del ruido. Por eso agregamos el test pareado.
- **Las anotaciones están sesgadas:** las hicimos nosotros, con un estilo de consulta propio (largas, sin tildes) y **sabiendo de qué tratan los libros**, no solo lo que dicen sus sinopsis. En Q02 (Jane Austen) las sinopsis no mencionan bailes ni matrimonios de conveniencia: ningún modelo que lea el texto puede acertar, y la métrica lo cuenta como error del modelo.
- **Costo, latencia y diversidad de resultados:** no los mide.

También encontramos un error en nuestra propia evaluación. Al ordenar con `argsort` todos los libros, incluso los de similitud 0, TF-IDF "acertaba" Q12 por el desempate por id (Robin Hood tiene id 2). Lo corregimos: un libro con similitud 0 no cuenta como recuperado.

4. Un caso donde la búsqueda falló

Q12: "forajido justiciero arquero que despoja a tiranos feudales para auxiliar desamparados" → *Robin Hood*.

La diseñamos para que no compartiera ninguna palabra con la sinopsis ("legendario héroe… robaba a gobernantes… para entregar parte del botín a necesitados"). Como se esperaba, TF-IDF no recupera nada. Pero **SBERT pone a Robin Hood en el puesto 12 y SBW en el 9**.

Hipótesis 1, truncamiento: descartada. La sinopsis supera los 128 tokens, pero la frase que describe al personaje está dentro de lo que el modelo ve.

Hipótesis 2, vocabulario raro: confirmada. Para evitar repetir palabras usamos términos poco frecuentes, y el tokenizer de SBERT los parte en fragmentos sin significado (`fora ##ji ##do`, `des ##po ##ja`, `desa ##mpa ##rados`). Con la misma idea en palabras comunes, "un bandido que roba a los ricos para darle a los pobres", que tampoco comparte ninguna palabra con la sinopsis, **SBERT pone a Robin Hood primero**.

Esas variantes las usamos solo para diagnosticar y no entran en la métrica. La conclusión es que nuestra consulta "sin solapamiento" medía también **vocabulario raro**, y eso perjudica a cualquier modelo. Al diseñar este tipo de consulta hay que usar sinónimos de uso frecuente.

Uso de asistentes de IA

Usamos asistentes de IA para escribir y depurar código (carga en pgvector, funciones SQL, gráficos) y para revisar el trabajo contra la consigna, lo que nos ayudó a encontrar errores en versiones intermedias. Las anotaciones de relevancia de `queries.json` las hicimos nosotros leyendo las sinopsis, y todas las cifras de este informe salen de las salidas del notebook `TP2_marquardt_aguirre_venesia_zappala.ipynb`.